Machine Learning with Big Data

Assignment 2

Efficient Similarity Detection using k-Grams, MinHash and LSH

Submitted by - Akshay Kumar (M25DE1028)

GitHub Link – Assignment 2

Step 1: Reading Documents

The four documents were read using Python file handling. Trailing newline characters were removed using .strip().

```
Document Lengths:
D1: 1749
D2: 1747
D3: 2132
D4: 1435
```

k-Gram Construction

Character 2-grams, character 3-grams, and word 2-grams were generated for all documents using a sliding window approach.

For a document of length n, the number of possible character k-grams is:

$$n - k + 1$$

Duplicates were removed using a set to ensure proper Jaccard computation.

Comparison of k-gram Representation

Character 2-grams capture very small fragments and are highly common across documents, which may artificially inflate similarity.

Character 3-grams preserve more contextual structure and provide better discrimination between documents.

Word 2-grams capture phrase-level similarity and provide more semantic meaning compared to character-level shingles.

Word 2-grams

Word-level 2-grams were constructed by splitting each document into words and forming consecutive word pairs.

As before, duplicates were removed using a set.

The number of unique word 2-grams for each document is shown in the output.

```
Unique Word 2-grams:
D1: 279
D2: 278
D3: 337
D4: 232
```

Step 2: Exact Jaccard Similarity Results

The computed Jaccard similarities for all document pairs are shown in Table 1.

Table 1: Exact Jaccard Similarity for All Document Pairs

	Pair	Char 2-gram	Char 3-gram	Word 2-gram
0	D1-D2	0.9811	0.9780	0.9408
1	D1-D3	0.8157	0.5804	0.1823
2	D1-D4	0.6444	0.3051	0.0302
3	D2-D3	0.8000	0.5680	0.1737
4	D2-D4	0.6413	0.3059	0.0303
5	D3-D4	0.6530	0.3121	0.0161

Step 3: Preparing Data for MinHash

All unique character 3-grams across the four documents were collected.

Each unique 3-gram was assigned a unique integer ID to enable hashing operations.

Total unique 3-grams = 1301.

```
Total unique 3-grams across all documents: 1301
```

Why Minimum Hash Value is Used

MinHash relies on the property that the probability of two sets having the same minimum hash value under a random permutation is equal to their Jaccard similarity.

Therefore, by generating multiple hash functions and comparing minimum hash values, we obtain an unbiased estimator of Jaccard similarity.

MinHash Estimation (t = 20)

Using 20 hash functions, the estimated Jaccard similarity between D1 and D2 was computed by comparing matching signature positions.

The estimated similarity is: 1.0

```
Length of signature for D1: 20
Estimated Jaccard Similarity (t=20): 1.0
```

Observation (t = 20)

The exact Jaccard similarity between D1 and D2 using character 3-grams is 0.978, indicating that the two documents are almost identical.

Using MinHash with t = 20 hash functions, the estimated similarity was 1.0.

Since the true similarity is very high, obtaining 20/20 matching signature values is statistically plausible.

This demonstrates that MinHash closely approximates Jaccard similarity even with a small number of hash functions when documents are highly similar.

MinHash Estimation for Different Values of t

The approximate Jaccard similarity between D1 and D2 was computed using different numbers of hash functions.

	t	Estimated Similarity
0	20	0.9500
1	60	0.9667
2	150	1.0000
3	300	0.9767
4	600	0.9700

Effect of Number of Hash Functions

1. The exact Jaccard similarity between D1 and D2 using character 3-grams is **0.978**. As the number of hash functions (t) increases, the MinHash estimate converges toward this true similarity value.
2. For smaller values of t (e.g., t = 20), the estimate shows higher variance and deviates more from the true similarity due to randomness in hashing.
3. For larger values of t (e.g., t = 300 and 600), the estimate stabilizes near **0.97–0.98**, closely matching the true Jaccard similarity, demonstrating improved estimation accuracy.

Choosing an Appropriate Value of t

1. Experimental results show that increasing t reduces variance and improves the stability of the MinHash estimate.
2. Although t = 150 produced an estimate of 1.0, this is a result of probabilistic fluctuation rather than improved accuracy.
3. t = 300 produced an estimate of **0.9767**, which is closest to the true similarity while maintaining reasonable computational cost, making it a suitable trade-off between accuracy and efficiency.

Step 4: Locality Sensitive Hashing (LSH)

Choosing r and b for LSH

Given t = 160 hash functions, multiple combinations of (r, b) were evaluated such that:

$$b \times r = 160$$

At threshold τ = 0.7, the probability:

$$f(s) = 1 - (1 - s^r)^b$$

was computed.

Among tested configurations, r = 8 and b = 20 produced a probability of approximately 0.695 at s = 0.7, providing good separation between similar and dissimilar document pairs.

Configurations with smaller r (e.g., r = 5) resulted in overly high probabilities, increasing false positives, while larger r values resulted in overly strict filtering and increased false negatives.

Therefore, r = 8 and b = 20 were selected.

```
r=5, b=32, Probability at s=0.7: 0.9972
r=8, b=20, Probability at s=0.7: 0.695
r=10, b=16, Probability at s=0.7: 0.3677
r=16, b=10, Probability at s=0.7: 0.0327
r=20, b=8, Probability at s=0.7: 0.0064
```

LSH Probability Results

Using r = 8 and b = 20, the LSH detection probability was computed for all document pairs.

Highly similar documents (e.g., D1–D2) have probability close to 1, while less similar pairs have significantly lower probabilities.

This demonstrates the effectiveness of LSH in separating similar and dissimilar document pairs.

	Pair	Exact Jaccard (3-gram)	LSH Probability
0	D1-D2	0.9780	1.0000
1	D1-D3	0.5804	0.2282
2	D1-D4	0.3051	0.0015
3	D2-D3	0.5680	0.1959
4	D2-D4	0.3059	0.0015
5	D3-D4	0.3121	0.0018

LSH Probability Analysis

Using r = 8 and b = 20, the LSH detection probability was computed for all document pairs.

The highly similar pair D1–D2 (Jaccard = 0.978) has a detection probability of 1.0, ensuring it is selected as a candidate pair.

Pairs with similarity below the threshold τ = 0.7 show significantly lower probabilities (≤ 0.23), demonstrating effective separation.

Low-similarity pairs (~0.30) have near-zero probability (~0.0015), indicating that LSH successfully filters dissimilar documents.

This confirms that the chosen (r, b) configuration produces a well-defined S-curve with strong separation at τ = 0.7.

Step 5: MinHash on MovieLens 100K Dataset

Loading the Dataset

The MovieLens 100K dataset was loaded using pandas. The file u.data contains user IDs, movie IDs, ratings, and timestamps.

For this experiment, only user IDs and movie IDs were used.

	user_id	movie_id	rating	timestamp
0	196	242	3	881250949
1	186	302	3	891717742
2	22	377	1	878887116
3	244	51	2	880606923
4	166	346	1	886397596

Exact User Similarity Results

The exact Jaccard similarity was computed for all user pairs (approximately 443,653 comparisons). The computation took approximately 3.7 seconds.

A total of 10 user pairs were found to have similarity ≥ 0.5.

These pairs serve as the ground truth for evaluating the MinHash approximation.

MinHash Evaluation (t = 50)

Using 50 hash functions:

```
True Positives: 4
False Positives: 65
False Negatives: 6
```

Although MinHash reduces computation time significantly, using only 50 hash functions results in high variance and unstable similarity estimates. This leads to a large number of false positives and missed true similar pairs.

This demonstrates that a small signature size is insufficient for accurate similarity detection in large datasets.

MinHash Evaluation (t = 100)

Using 100 hash functions:

```
Results for t = 100
True Positives: 5
False Positives: 17
False Negatives: 5
```

Compared to t = 50, the number of false positives decreased significantly, and the detection accuracy improved.

Increasing the signature length reduces variance in similarity estimation and leads to better performance.

MinHash Evaluation (t = 200)

Using 200 hash functions:

```
Results for t = 200
True Positives: 9
False Positives: 8
False Negatives: 1
```

Compared to t = 100, the number of false positives further decreased and the number of true positives increased significantly. Only one true similar pair was missed.

Increasing the signature length to 200 greatly reduces variance in similarity estimation and produces results that closely match the exact Jaccard similarity, demonstrating strong accuracy with reasonable computational cost.

MinHash Evaluation on MovieLens Dataset

MinHash was evaluated using signature sizes of 50, 100, and 200 hash functions.

t	True Positives	False Positives	False Negatives
50	4	65	6
100	5	17	5
200	9	8	1

With t = 50, the signature was too small, leading to high variance and many incorrect predictions.

Increasing t to 100 significantly reduced false positives.

With t = 200, the MinHash approximation closely matched the true similarity, detecting 9 out of 10 similar user pairs while maintaining a low number of false positives.

This demonstrates that increasing signature size improves estimation accuracy at the cost of additional computation.

MinHash Evaluation (t = 50, Averaged over 5 Runs)

Average results over 5 independent runs:

```
Average Results (t=50)
Avg TP: 6.8
Avg FP: 162.2
Avg FN: 3.2
```

Although MinHash detects more true similar pairs on average, the number of false positives increases significantly due to high variance in small signatures. This indicates that 50 hash functions are insufficient for stable similarity estimation on large datasets.

MinHash Evaluation (t = 100, Averaged over 5 Runs)

Average results over 5 independent runs:

```
Average Results (t=100)
Avg TP: 8.0
Avg FP: 16.4
Avg FN: 2.0
```

Compared to t = 50, increasing the signature size significantly reduced false positives and improved detection accuracy. The MinHash estimates became more stable and closely aligned with exact similarity results.

MinHash Evaluation (t = 200, Averaged over 5 Runs)

Average results over 5 independent runs:

```
Average Results (t=200)
Avg TP: 8.6
Avg FP: 13.2
Avg FN: 1.4
```

Compared to t = 100, increasing the signature size further improves detection stability. The number of false positives decreases slightly, and false negatives reduce further, resulting in better overall accuracy.

This demonstrates that larger signature sizes reduce estimation variance and produce results that closely approximate exact Jaccard similarity, though with increased computational cost.

Overall Analysis of MinHash Performance

The performance of MinHash improves consistently as the number of hash functions increases.

With t = 50, the signature size is too small, leading to high variance and a large number of false positives.

At t = 100, performance improves significantly, with reduced false positives and better detection of true similar pairs.

At t = 200, the MinHash approximation closely matches the exact similarity results, achieving high true positive detection while maintaining relatively low false positives and false negatives.

These results demonstrate that increasing signature length improves estimation stability and accuracy, though at the cost of increased computational effort.

	t (Hash Functions)	Avg True Positives	Avg False Positives	Avg False Negatives
0	50	6.8	162.2	3.2
1	100	8.0	16.4	2.0
2	200	8.6	13.2	1.4

LSH Evaluation (t = 50, r = 5, b = 10, threshold = 0.6)

```
LSH Results (t=50, r=5, b=10, threshold=0.6)
True Positives: 3
False Positives: 686
False Negatives: 0
```

LSH successfully detected all truly similar user pairs (perfect recall). However, due to the small signature size and aggressive banding, a large number of false positives were generated.

This demonstrates that while LSH significantly reduces the number of comparisons, small signature sizes can lead to excessive candidate generation.

LSH Evaluation (t = 100, r = 5, b = 20, threshold = 0.6)

```
LSH Results (t=100, r=5, b=20, threshold=0.6)
True Positives: 3
False Positives: 1329
False Negatives: 0
```

The LSH configuration successfully detected all truly similar user pairs (perfect recall). However, increasing the number of bands significantly increased false positives.

This demonstrates that LSH candidate generation is highly sensitive to band configuration. Larger numbers of bands increase collision probability, resulting in more candidate pairs.

LSH Evaluation (t = 200, r = 5, b = 40, threshold = 0.6)

```
LSH Results (t=200, r=5, b=40, threshold=0.6)
True Positives: 3
False Positives: 1935
False Negatives: 0
```

This configuration successfully detected all true similar pairs but produced an extremely large number of false positives.

Increasing the number of bands while keeping r small significantly increases collision probability, leading to excessive candidate generation.

This demonstrates that improper band configuration can drastically reduce precision.

LSH Configuration Comparison (Threshold = 0.6)

Increasing the number of rows per band (r) significantly reduces false positives but may introduce false negatives.

The configuration (t = 200, r = 10, b = 20) produced:

```
LSH Results (t=200, r=10, b=20, threshold=0.6)
True Positives: 2
False Positives: 6
False Negatives: 1
```

Although one true pair was missed, this configuration dramatically reduced candidate pairs and false positives, demonstrating a better precision–recall balance.

This highlights the importance of carefully selecting LSH parameters based on the desired trade-off between recall and precision.

Overall Analysis of LSH (Threshold = 0.6)

The LSH performance strongly depends on the choice of band parameters (r, b).

Configurations with small r (e.g., r = 5) achieved perfect recall (no false negatives) but produced a very large number of false positives due to excessive band collisions.

Increasing r to 10 significantly reduced false positives while maintaining strong detection performance, although one true pair was missed.

The configuration (t = 200, r = 10, b = 20) achieved the best balance between precision and recall, dramatically reducing candidate pairs while maintaining high detection accuracy.

This demonstrates the fundamental trade-off in LSH between recall and precision controlled by the band structure.

	Configuration	True Positives	False Positives	False Negatives
0	t=50, r=5, b=10	3	686	0
1	t=100, r=5, b=20	3	1329	0
2	t=200, r=5, b=40	3	1935	0
3	t=200, r=10, b=20	2	6	1

LSH Evaluation (Threshold = 0.8)

Only one user pair had similarity ≥ 0.8.

All LSH configurations successfully detected this pair (no false negatives).

However, configurations with small r (e.g., r = 5) produced a large number of false positives due to excessive band collisions.

The configuration (t = 200, r = 10, b = 20) dramatically reduced false positives to only 7 while still detecting the true pair.

This demonstrates that increasing rows per band improves precision without harming recall for highly similar pairs.

	Configuration	True Positives	False Positives	False Negatives
0	t=50, r=5, b=10	1	688	0
1	t=100, r=5, b=20	1	1331	0
2	t=200, r=5, b=40	1	1937	0
3	t=200, r=10, b=20	1	7	0